



UssdGatewayTz

PARTNER INTEGRATION GUIDE

Version: 1.0

Audience: External partners hosting a USSD handler URL

Owner: UssdGatewayTz operations team

CLASSIFIED AS CONFIDENTIAL — BUSINESS PARTNER

Contents

1. **Architecture** — the request/response loop in one diagram

2. **Onboarding checklist** — what you give us to go live

3. **Request shape** — the unified JSON we POST to you

4. **Reply shape** — CON / END, JSON or plain text

5. **Lifecycle** — which events you will and won't see

6. **Authentication** — none vs bearer

7. **Timeouts, retries, error handling**

8. **Per-MNO quirks** — Vodacom, Airtel, Tigo, Halotel

9. **Reference handlers** — Flask & Express

0. **Testing** — curl the gateway contract

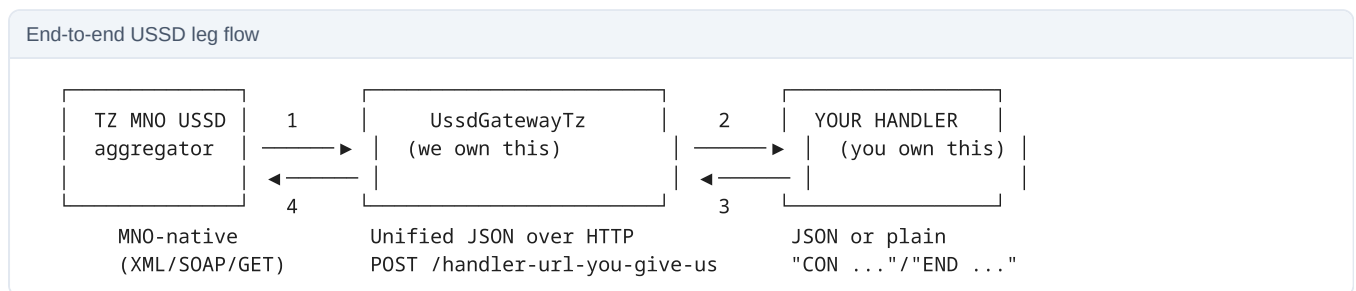
1. **Reports & reconciliation**

2. **FAQ**

3. **Operational contacts**

This document is for external partners hosting a handler that UssdGatewayTz forwards live USSD traffic to. After onboarding, the gateway accepts USSD requests from the Tanzanian MNOs (Vodacom, Airtel, Tigo / Yas, Halotel), normalises them into a unified JSON shape, and POSTs them to your handler URL. Your handler replies with the menu/result and the gateway returns that to the MNO.

1. Architecture, in one diagram



1. MNO pushes a USSD leg (every screen the user sees = one leg).
2. Gateway parses the MNO-native shape, applies routing, **POSTs unified JSON to your handler**.
3. Your handler replies — either `{ "action": "CON|END", "message": "..."}` or plain text starting with `CON / END`.
4. Gateway translates your reply back to the MNO's native protocol.

Each USSD session typically consists of 2–6 legs (dial → menu → input → submenu → input → result). Every leg is one HTTP round-trip to your handler.

2. Onboarding checklist

To onboard a shortcode you provide us with:

WHAT	WHY	EXAMPLE
Shortcode / partner slug	Routing key. For Vodacom + Halotel = the dialed USSD code (e.g. <code>*123#</code>). For Airtel + Tigo = the partner-slug the MNO assigns to your URL (e.g. <code>glpair</code> , <code>glptigo</code>).	<code>*123#</code> or <code>glptigo</code>
MNO	Which network — one shortcode row per (MNO, code) pair.	<code>vodacom</code>
Handler URL	Where we POST traffic. HTTPS required in production.	<code>https://api.example.com/ussd</code>
Auth mode	<code>none</code> if you trust the inbound IP (VPN tunnel or IP-allowlisted), or <code>bearer</code> for a static token in the <code>Authorization</code> header.	<code>bearer</code>
Bearer token (only if <code>auth_mode=bearer</code>)	Long random string. Rotate by emailing ops + redeploying.	64-char base64
Per-leg timeout	How long we wait for your handler reply. Default 5s; max 10s. Stay well under the MNO's session budget.	<code>5</code>
Owner email	The portal user that sees this shortcode's reports + can manage it.	<code>ops@example.com</code>

Once provisioned, MNO traffic to your code/slug starts flowing immediately.

3. Request shape — what we POST to your handler

Gateway → handler request

```
POST https://api.example.com/usdd
Content-Type: application/json
Authorization: Bearer <your-token> # only if auth_mode=bearer

{
  "operator": "vodafone",
  "msisdn": "255712345678",
  "session_id": "ABC123",
  "service_code": "*123#",
  "ussd_string": "1*2",
  "event": "start",
  "raw_payload": { ... full MNO-native payload ... }
}
```

Field reference

FIELD	TYPE	NOTES
operator	"vodafone" "airtel" "tigo" "halotel"	Canonical MNO key. Always present.
msisdn	string	International, no leading + (e.g. 255712345678). The subscriber's number.
session_id	string	MNO-issued session identifier. Stable across every leg of one USSD session. Use this as the key in your per-user session state.
service_code	string	What the user dialed (*123#) OR the partner slug (glptigo) — matches what was onboarded. Stable across legs.
ussd_string	string	Accumulated menu trail with * separators. Empty on event=start ; grows on each input leg as "1" , then "1*2" , then "1*2*9" , etc.
event	"start" "input"	start = the first leg (the dial). input = every subsequent leg (the user pressed something on the previous menu).
raw_payload	object	The original MNO-native payload — useful for forensics / MNO-specific fields not in our unified shape. Don't rely on its structure (different per MNO).

Handler must be idempotent on (session_id, ussd_string)

USSD aggregators occasionally retransmit. If you receive the same (session_id, ussd_string) twice within a few seconds, treat the second as a retry and return the same answer — **don't double-charge / double-write**.

4. Reply shape — what your handler returns

You can return **either** of two formats. Pick whichever fits your stack; the gateway accepts both.

4a. JSON (preferred for new integrations)

CON — continue, prompt for input

```
HTTP/1.1 200 OK
Content-Type: application/json

{ "action": "CON", "message": "Choose option:\n1. Account balance\n2. Buy airtime" }
```

END — terminate session

```
HTTP/1.1 200 OK
Content-Type: application/json

{ "action": "END", "message": "Your balance: 1,250 TZS" }
```

4b. Plain text (matches existing aggregator conventions)

CON — plain text

```
HTTP/1.1 200 OK
Content-Type: text/plain

CON Choose option:
1. Account balance
2. Buy airtime
```

END — plain text

```
HTTP/1.1 200 OK
Content-Type: text/plain

END Your balance: 1,250 TZS
```

Action meaning

ACTION	WHAT THE USER SEES	SESSION CONTINUES?
CON	Your message displayed; input box appears below for the next leg.	yes — gateway expects another inbound leg with event="input"
END	Your message displayed; no input box . Session terminates after they read it.	no — final leg

Message limits

USSD frames have a hard limit of **~182 characters** (depends on MNO/handset). Keep messages short, count newlines as characters. Long messages get truncated by the MNO — test on a real device before going live.

5. Lifecycle — what `event` will I see?

The gateway only ever forwards two lifecycle events to your handler:

EVENT	WHEN	WHAT USSD_STRING IS
<code>start</code>	First leg — the user just dialed your code	<code>" "</code> (empty)
<code>input</code>	Subsequent leg — the user pressed something on the previous menu	accumulated trail (e.g. <code>"1"</code> , <code>"1*2"</code> , <code>"1*2*9"</code>)

You will **NEVER** see:

- `user_cancelled` — the user pressed Cancel. The gateway acks the MNO directly and your session is silently gone.
- `timeout` — the MNO closed the session (took too long between legs).
- `charge_failed` (Vodacom premium-rate) — Vodacom's billing said no.
- Halotel's `display_ack` (type=103) — informational; no need to act.

If you maintain server-side session state keyed by `session_id`, expire it after **~2 minutes of inactivity** (USSD sessions live ~60–90s typically). You won't get notified when a session ends without an `END` from you.

6. Authentication

MODE	WHEN TO USE	WHAT WE SEND
<code>none</code>	Your handler is on a private VPN + IP-allowlist that only we can reach.	No <code>Authorization</code> header.
<code>bearer</code>	Public handler (or layered defence over network controls).	<code>Authorization: Bearer <your-token></code> — exact string match. Token never rotates without your action; we don't auto-rotate.

To rotate a bearer token: email ops with the new value; we'll update the DB row at a coordination time. Plan for a brief window where both old and new tokens are accepted on your side.

We do **not** sign request bodies (no HMAC). If body integrity matters to you, terminate TLS on your handler and trust the connection.

7. Timeouts, retries, error handling

Timeouts

- We wait **at most** `timeout_secs` (per-shortcode, default 5s) for your reply.
- If you don't respond in time, the gateway sends the MNO an `END Service unavailable` and logs the leg with `error_class="timeout"`.
- Plan for ≤ 3s p99 latency** in your handler. The MNO's overall session budget is ~10s; we need to leave headroom for the round-trip back to them.

Retries

USSD has **no retry semantics**. We do **not** retry your handler if it errors or times out — the user sees `Service unavailable` and the session is over. If your handler is briefly unhealthy, sessions during that window are lost. There's no make-up later. Build for low-latency, high-availability.

What happens on each error class

HANDLER RETURNS	WHAT THE USER SEES	LOGGED ERROR_CLASS
HTTP 4xx / 5xx	END Service unavailable. Please try later.	non2xx
Connection timeout / refused	same	transport
Reply that isn't valid CON / END JSON or plain text	same	unparseable
200 OK with valid CON ... / END ...	your message	(no error)

All errors are logged with the full `raw_payload`, your handler URL, and the elapsed time. The shortcode owner sees these on the dashboard immediately.

8. Per-MNO quirks worth knowing

These are all handled by the gateway; your handler sees the unified shape. They're documented here so you understand what arrives:

Vodacom TruRoute XML

- MNO uses XML POST or GET to us; gets translated to JSON for you.
- Premium charging (`type=10`) is filtered out before reaching you.
- `service_code` is the dialed code (`*123#`).

Airtel

- Per-partner URLs at the MNO (your slug `glpair` is the routing key on our side).
- No first-leg flag from the MNO — we detect via session-cache presence.
- `service_code` is your partner slug (e.g. `glpair`).
- The actual dialed USSD (e.g. `*147*03#`) appears in `raw_payload.dialed_code` on the `start` leg only.

Tigo (Yas)

- Two parameter schemes in concurrent use at the MNO (we abstract both). The dialed code appears in `raw_payload.dialed_code` .
- `service_code` is your partner slug (e.g. `glptigo`).

Halotel

- The only MNO with **bidirectional SOAP**: our reply to your `CON / END` is pushed back to Halotel on a separate HTTP call (not the same response cycle). Transparent to you.
- If you reply `END` and the outbound push fails (Halotel transient outage), the customer's USSD session times out on Halotel's side; we cannot recover within the session window.

9. Reference handler implementations

Python (Flask)

handler.py

```
from flask import Flask, request, jsonify

app = Flask(__name__)
SESSIONS = {} # in-memory; use Redis in production

@app.post("/ussd")
def ussd():
    # Optional: validate bearer token
    # if request.headers.get("Authorization") != f"Bearer {TOKEN}":
    #     return ("", 401)

    body = request.get_json(force=True)
    session_id = body["session_id"]
    msisdn = body["msisdn"]
    ussd_string = body["ussd_string"]
    event = body["event"]

    # Branch the menu by the accumulated input trail.
    if event == "start":
        SESSIONS[session_id] = {"started_at": time.time()}
        return jsonify(action="CON",
                        message="Welcome!\n1. Balance\n2. Buy airtime")

    trail = ussd_string.split("*")
    if trail == ["1"]:
        balance = lookup_balance(msisdn)
        return jsonify(action="END", message=f"Your balance: {balance} TZS")

    if trail == ["2"]:
        return jsonify(action="CON", message="Enter amount in TZS:")

    if trail[0] == "2" and len(trail) == 2:
        amount = int(trail[1])
        # ...charge logic...
        return jsonify(action="END", message=f"Bought {amount} TZS.")

    return jsonify(action="END", message="Invalid option")

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8081)
```

Node.js (Express)

```
const express = require("express");
const app = express();
app.use(express.json());

const sessions = new Map(); // use Redis in production

app.post("/ussd", (req, res) => {
  const { session_id, msisdn, ussd_string, event } = req.body;

  if (event === "start") {
    sessions.set(session_id, { startedAt: Date.now() });
    return res.json({ action: "CON",
      message: "Welcome!\n1. Balance\n2. Buy airtime" });
  }

  const trail = ussd_string.split("*");
  if (trail.join("*") === "1") {
    return res.json({ action: "END",
      message: `Your balance: ${lookupBalance(msisdn)} TZS` });
  }
  if (trail.join("*") === "2") {
    return res.json({ action: "CON", message: "Enter amount in TZS:" });
  }
  if (trail[0] === "2" && trail.length === 2) {
    const amount = parseInt(trail[1], 10);
    // ...charge logic...
    return res.json({ action: "END", message: `Bought ${amount} TZS.` });
  }
  return res.json({ action: "END", message: "Invalid option" });
});

app.listen(8081, "0.0.0.0");
```

10. Testing against a non-production gateway

Once you've stood up your handler, ask ops to point a sandbox shortcode (e.g. `*150*99#` on Vodacom) at your URL. You can then dial from any test SIM. Or, simulate the gateway directly with curl:

Simulate the START leg

```
curl -sS -X POST https://api.example.com/usssd \
-H 'Content-Type: application/json' \
-H 'Authorization: Bearer your-token' \
-d '{
  "operator":      "vodacom",
  "msisdn":        "255700000001",
  "session_id":    "TEST-SESSION-1",
  "service_code":  "*150*99#",
  "ussd_string":   "",
  "event":         "start",
  "raw_payload":   {}
}'
```

Simulate the INPUT leg "1"

```
curl -sS -X POST https://api.example.com/usssd \
-H 'Content-Type: application/json' \
-H 'Authorization: Bearer your-token' \
-d '{
  "operator":      "vodacom",
  "msisdn":        "255700000001",
  "session_id":    "TEST-SESSION-1",
  "service_code":  "*150*99#",
  "ussd_string":   "1",
  "event":         "input",
  "raw_payload":   {}
}'
```

If both round-trips return well-formed `CON / END` JSON in under a second, you're ready for live traffic.

11. Reports + reconciliation

The owner email you provided gets a dashboard login. There they can:

- See every leg / session attributed to their shortcode (real time).
- Filter by date / MNO / msisdn / session_id / error class.
- See **per-MNO billable session counts** (`CEIL(duration / window)` — Vodacom 20s, Airtel/Tigo 30s, Halotel per-leg) so they can reconcile against the MNO invoice.
- Export filtered slices to CSV (queued; download link appears when ready).

If you need a service account that can reach the dashboard API directly (e.g. to pull daily totals into your own BI), email ops.

12. FAQ

Q: Why do I sometimes see `event="start"` with a non-empty `ussd_string` ?

A: You shouldn't. `start` always carries an empty `ussd_string` . If you do see it, treat it as a rare aggregator retransmit and respond normally.

Q: My handler is fast but I'm seeing `error_class="timeout"` on the dashboard.

A: Check the network path — DNS, TLS handshake, and our egress latency all count toward the timeout. Ask ops to raise `timeout_secs` on your shortcode if the round-trip genuinely needs >5s, but that eats into the MNO's session budget.

Q: How do I correlate dashboard logs with my own logs?

A: Use `session_id` — it's MNO-issued and stable across every leg. We log it on every row; you should log it on every handler entry.

Q: What if I need to send multiple SMS / push notifications as part of the USSD flow?

A: Send them asynchronously AFTER replying `END` — don't block the handler response on slow side-effects. The USSD session is over once you reply; the user is gone from the USSD screen and won't see anything you do afterwards.

Q: How do I deactivate a shortcode temporarily?

A: Email ops or use the dashboard if you're a `super_admin`. Soft-deactivation (`active=false`) means we return `Service not configured` to anyone who dials it; reactivating restores routing immediately. Historical reports are preserved either way.

Q: Do you support charging (premium-rate USSD)?

A: Currently the gateway forwards Vodacom premium-charge failure notifications but does NOT initiate charges itself. If you need premium billing, raise it with ops — it requires extending the `UnifiedReply` contract with a `premium` block (Vodacom TruRoute spec exposes the hook; not yet wired through).

13. Operational contacts

PURPOSE	CONTACT
Onboarding / config changes	<i>[partner-ops@example.com]</i>
Production incidents	<i>[oncall@example.com]</i>
Security / token rotation	<i>[security@example.com]</i>

Owners — replace these placeholders with your real distribution lists before sharing externally.

Related docs. The companion runbooks live in the repo under `docs/` — `partner-quickstart.md` , the machine-readable `openapi.yaml` , and the per-MNO notes (`mno-vodacom-truroute.md` , `mno-airtel.md` , `mno-tigo.md` , `mno-halotel.md`).